

Recherche Opérationnelle

Étude de Trois Problèmes

Yahya Sidaty El Hachmy

C26698

MIAGE – Université de Nouakchott

2025-2026

Plan

- 1 Introduction
- 2 Optimisation du transport
- 3 Files d'attente hospitalières
- 4 Heuristique du TSP
- 5 Conclusion

Définition

La recherche opérationnelle utilise des modèles mathématiques afin d'optimiser les systèmes réels et améliorer la prise de décision.

Applications :

- Transport
- Hôpitaux
- Livraison
- Logistique

Définition du problème

Le problème de transport consiste à distribuer des marchandises entre plusieurs usines et entrepôts tout en minimisant le coût total.

Objectif

Réduire les coûts logistiques.

Capacités

$$U1 = 100$$

$$U2 = 150$$

Demandes

$$E1 = 80$$

$$E2 = 120$$

$$E3 = 50$$

Tableau des coûts

	E1	E2	E3
U1	4	6	8
U2	5	3	7

$$\min Z = 4x_{11} + 6x_{12} + 8x_{13} + 5x_{21} + 3x_{22} + 7x_{23}$$

x_{ij} représente les quantités transportées.

Résultats obtenus

Variables

$$x_{11} = 80$$

$$x_{12} = 20$$

$$x_{22} = 100$$

$$x_{23} = 50$$

Coût total

$$Z = 1090$$

```
model = LpProblem("Transport", LpMinimize)

x11 = LpVariable('x11', lowBound=0)
x12 = LpVariable('x12', lowBound=0)
x13 = LpVariable('x13', lowBound=0)

x21 = LpVariable('x21', lowBound=0)
x22 = LpVariable('x22', lowBound=0)
x23 = LpVariable('x23', lowBound=0)

model += 2*x11 + 4*x12 + 5*x13 + 3*x21 + 1*x22 + 7*x23

model += x11 + x12 + x13 <= 100
model += x21 + x22 + x23 <= 150
model += x11 + x21 == 80
model += x12 + x22 == 120
model += x13 + x23 == 50

model.solve()

for v in model.variables():
    print(v.name, "=", v.varValue)

print("Cost =", value(model.objective))
```

- Réduction des coûts
- Satisfaction des demandes
- Optimisation des ressources
- Meilleure distribution

Les files d'attente hospitalières permettent d'améliorer l'organisation des patients dans les hôpitaux.

Objectif

Réduire le temps d'attente.

$$\lambda = 12$$

$$\mu = 5$$

$$c = 3$$

- λ : taux d'arrivée
- μ : taux de service
- c : nombre de médecins

$$\rho = \frac{\lambda}{c\mu}$$

$$\rho = \frac{12}{15}$$

$$\rho = 0.8$$

```
def p0(lam, mu, c):
    somme = 0

    for n in range(c):
        somme += (lam / mu) ** n / math.factorial(n)

    dernier = ((lam / mu) ** c) / (math.factorial(c) * (1 - rho))

    return 1 / (somme + dernier)

P0 = p0(lam, mu, c)

Lq = (
    P0
    * ((lam / mu) ** c)
    * rho
    / (math.factorial(c) * (1 - rho) ** 2)
)

Wq = Lq / lam
W = Wq + (1 / mu)

print("Mod le M/M/c - Centre hospitalier")
print("Taux d'arriv e      =", lam, "patients/heure")
print("Taux de service     =", mu, "patients/heure")
print("Nombre de serveurs c =", c)
print("Utilisation         =", round(rho, 3))
print("Nombre moyen en attente Lq =", round(Lq, 3))
print("Temps moyen d'attente Wq =", round(Wq * 60, 2), "minutes")
print("Temps moyen dans le syst me =", round(W * 60, 2), "minutes")
```

- Réduction du temps d'attente
- Meilleure organisation
- Réduction de la congestion
- Amélioration des soins

Le problème du voyageur de commerce consiste à trouver le plus court chemin passant par toutes les villes une seule fois.

Objectif

Obtenir une solution proche de l'optimum rapidement.

Exemple

Distances utilisées :

$$AB = 10$$

$$BE = 12$$

$$ED = 16$$

$$DC = 30$$

$$CA = 15$$

- Méthode gloutonne
- Recuit simulé
- Recherche tabou

Chemin trouvé :

$$A \rightarrow B \rightarrow E \rightarrow D \rightarrow C \rightarrow A$$

Distance :

83

Distance optimale :

78

```
def distance(a, b):
    if a == b:
        return 0
    return distances.get((a, b), distances.get((b, a)))

depart = "A"
tour = [depart]
ville_actuelle = depart
distance_totale = 0

while len(tour) < len(villes):
    non_visitees = [v for v in villes if v not in tour]

    prochaine_ville = min(
        non_visitees,
        key=lambda ville: distance(ville_actuelle, ville)
    )

    distance_totale += distance(ville_actuelle, prochaine_ville)
    tour.append(prochaine_ville)
    ville_actuelle = prochaine_ville

distance_totale += distance(ville_actuelle, depart)

print("Tour trouv  par heuristique greedy :")
print(" -> ".join(tour + [depart]))
print("Distance totale :", distance_totale)
```

- Temps de calcul réduit
- Solution proche de l'optimum
- Applications logistiques modernes
- Optimisation des livraisons

Résumé

La recherche opérationnelle permet :

- d'optimiser les coûts
- d'améliorer les performances
- d'aider à la prise de décision

Merci pour votre attention